

Software Engineering Project
Assessment 4

Final Design Document

Blueberry — AI Dependency Validator

Team Members

Parthiban M — 24BYB0135
Nisha Rathi — 24BYB0137

Introduction

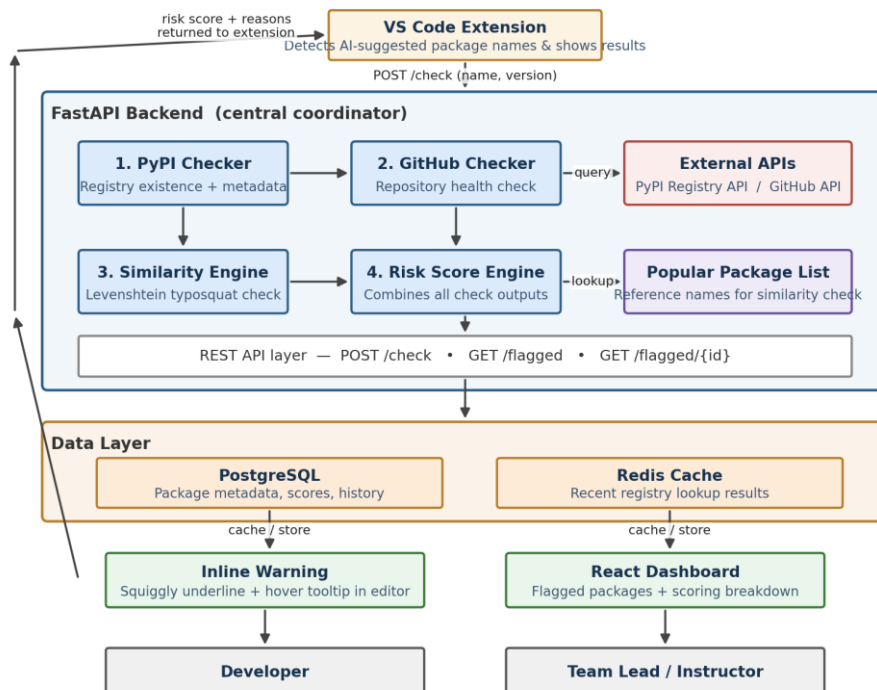
Blueberry is a VS Code extension and backend service that checks AI-generated package recommendations before a developer installs them. AI coding assistants such as GitHub Copilot, Cursor, Claude Code, Gemini Code Assist, and Codeium suggest package names by predicting likely text, not by confirming that the package actually exists, is maintained, or is safe. This can lead a developer to install a package that was never published, that is a lookalike of a trusted package (typosquatting), or that has none of the trust signals real packages have.

1. Final System Architecture

Blueberry is built as a thin client and a central backend. The VS Code extension only detects package names and sends them to the backend — it does not do any verification itself. All the real checking work happens inside the FastAPI backend, which keeps the extension small and fast, and keeps all the verification logic in one place that is easy to update.

When a package name is detected, the backend runs it through four checks, one after another: it confirms the package exists on PyPI, it checks whether the linked GitHub repository looks active and trustworthy, it compares the name against a list of popular packages to catch typosquatting, and finally it combines all three results into one risk score with plain-English reasons. The result is sent back to the extension, saved to the database, and can also be reviewed later in the dashboard.

Figure 1 — Final System Architecture of Blueberry



Component Descriptions

Component	Description
VS Code Extension	Runs inside the developer's editor. Watches for AI-suggested package names, sends them to the backend, and shows the result as an inline warning.
FastAPI Backend	The central coordinator. Exposes a REST API and runs the four verification checks for every package name it receives.
PyPI Checker	Confirms whether a package actually exists on PyPI and pulls its published metadata (publish date, version history, download count, repository link).
GitHub Checker	Looks at the package's linked GitHub repository and checks whether it is active, recently maintained, and consistent with the PyPI listing.
Similarity Engine	Compares the candidate name against a list of popular package names using Levenshtein distance, to catch names that are suspiciously close to a trusted package.
Risk Score Engine	Combines the outputs of the three checks above into one explainable, rule-based score with a list of plain-English reasons.
Popular Package List	A reference list of well-known, high-traffic package names used only by the Similarity Engine for comparison.
PostgreSQL	Stores package metadata, computed risk scores, and the history of flagged packages.
Redis Cache	Stores recent PyPI and GitHub lookup results so the same package is not re-checked against the outside world every time it is seen.
React Dashboard	A separate web page where a developer or team lead can review every flagged package and see exactly why it was flagged.
External Services	PyPI Registry API and GitHub API — two services outside the project's control that the backend depends on.

2. Detailed Module Descriptions

The system is broken into four groups of modules, matching the four layers in the architecture

VS Code Extension Modules

Module	Input	Output	Interacts With
Editor Event Listener	Editor events: inline completions, AI chat responses, terminal text	Raw text that may contain a package name	AST / Import Parser
AST / Import Parser	Raw code or command text	Candidate package name(s)	Package Candidate Extractor
Package Candidate Extractor	Candidate package name	A normalised check request (name, version, ecosystem)	Backend API Client, Local Cache
Backend API Client	Check request	Risk score response from the backend	FastAPI Backend (POST /check)
Local Cache	Check request	Cached risk score, if this package was checked recently	Backend API Client, Risk Response Handler
Risk Response Handler	Risk score + reasons from the backend	Updated local cache, a trigger to render the warning	Inline Decoration Renderer, Diagnostics Provider
Inline Decoration Renderer	Risk score + reasons	Squiggly underline and hover tooltip in the editor	VS Code editor UI
Diagnostics Provider	Risk score + reasons	An entry in VS Code's Problems panel	VS Code Problems API
Quick-Fix / Command Actions	Developer command (click or shortcut)	'View details', 'Ignore', or 'Open dashboard' action	React Dashboard, Risk Response Handler

FastAPI Backend Modules

Module	Input	Output	Interacts With
PyPI Checker	Package name	PackageMetadata, or a not-found result	PyPI Registry API, Risk Score Engine, Redis
GitHub Checker	Repository URL from the PyPI metadata	RepositoryHealth (active, last commit date, matches registry)	GitHub API, Risk Score Engine, Redis
Similarity Engine	Package name	SimilarityResult (closest match, edit distance, similarity %)	Popular Package List, Risk Score Engine
Risk Score Engine	PackageMetadata, RepositoryHealth, SimilarityResult	RiskScore (score, reasons, breakdown)	PostgreSQL, Redis, REST API layer

Data Layer Modules

Module	Input	Output	Interacts With
PostgreSQL Data Store	RiskScore records, FlaggedPackage records	Stored records; query results for the dashboard	Risk Score Engine, Dashboard Service
Redis Cache	Registry lookup results (PyPI / GitHub)	Cached results on repeat lookups	PyPI Checker, GitHub Checker

Dashboard Module

Module	Input	Output	Interacts With
Dashboard Service / React Dashboard	HTTP GET requests from the browser	List of flagged packages; full scoring breakdown for one package	PostgreSQL, Developer, Team Lead / Instructor

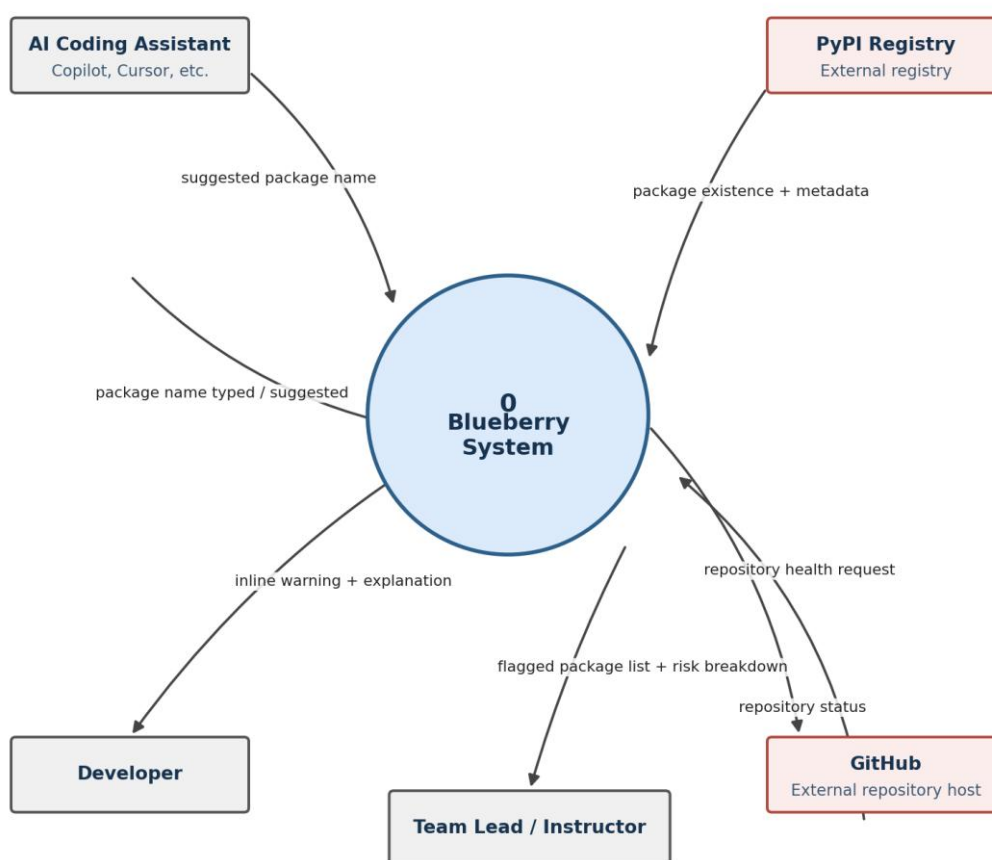
3. Data Flow Diagrams

Data flow diagrams show how information moves through Blueberry, without describing the internal code structure. The context diagram (Level 0) treats the whole system as a single process and shows who it exchanges data with. The Level-1 diagram then opens that single process up into its major steps.

Context Diagram (Level-0 DFD)

At the context level, Blueberry is one process. It receives package names from the AI coding assistant and the developer, calls out to PyPI and GitHub to verify them, and sends results back to the developer (as an inline warning) and to the team lead or instructor (as a dashboard view).

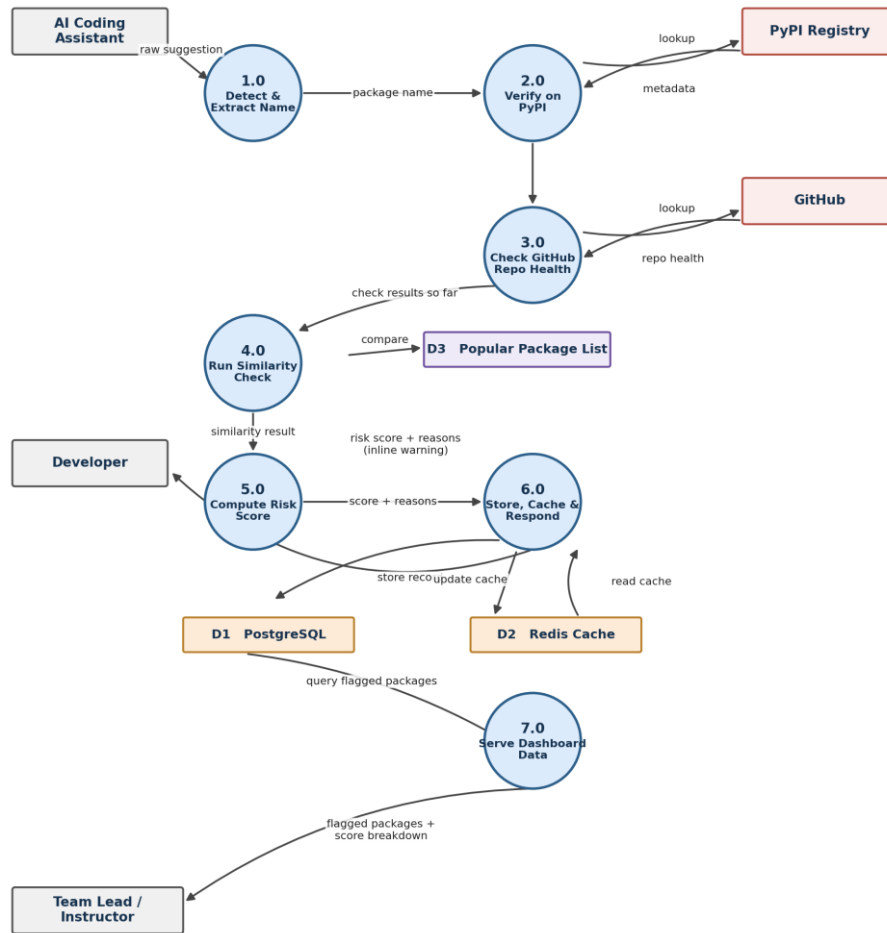
Figure 2 — Context Diagram (DFD Level 0)



Level-1 Data Flow Diagram

Breaking the single process open shows the seven steps a package name goes through: it is detected and extracted, verified on PyPI, checked on GitHub, compared against popular package names, scored, stored and cached, and finally served back to the extension or the dashboard.

Figure 3 — Level-1 Data Flow Diagram



Description of Each Process

#	Process	What it does
1.0	Detect & Extract Name	Watches the editor for AI-generated suggestions and pulls out a candidate package name.
2.0	Verify on PyPI	Asks the PyPI Registry whether the package exists and fetches its metadata.
3.0	Check GitHub Repo Health	Asks GitHub whether the linked repository is active and consistent with the registry listing.
4.0	Run Similarity Check	Compares the name against the popular package list to catch likely typosquats.
5.0	Compute Risk Score	Combines the three check results into one explainable score with reasons.
6.0	Store, Cache & Respond	Saves the result to PostgreSQL, updates the Redis cache, and returns the result to the extension.
7.0	Serve Dashboard Data	Answers dashboard requests with the list of flagged packages and their scoring breakdown.

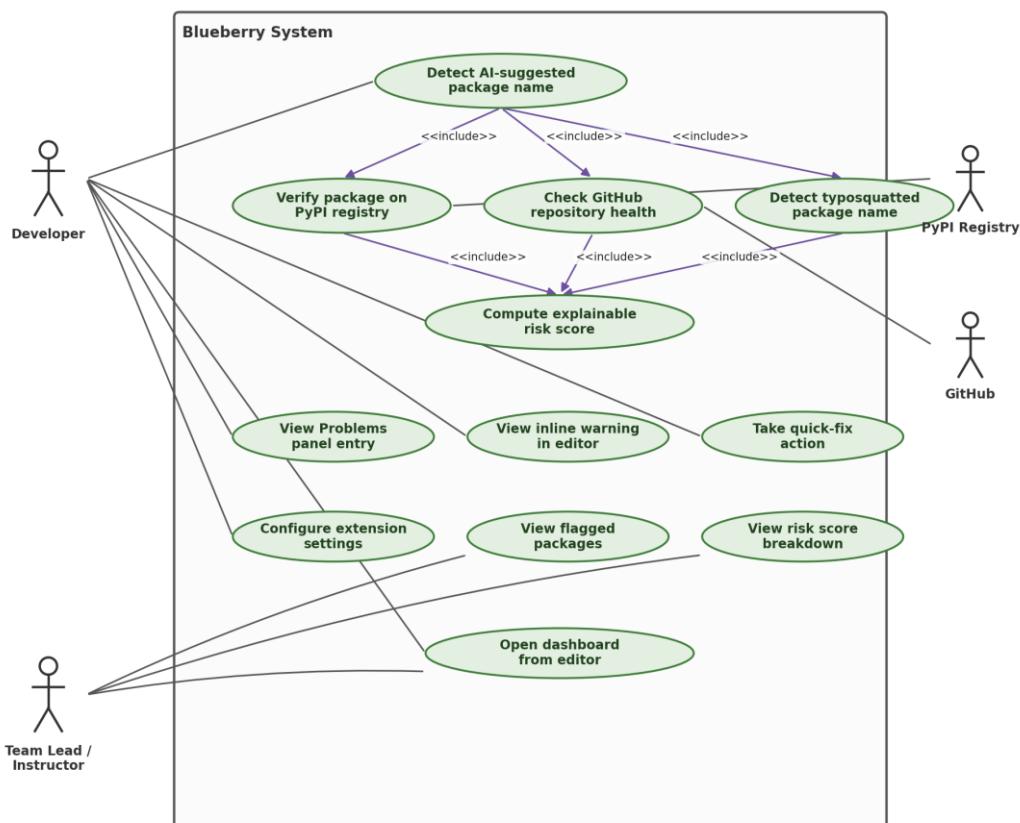
4. Complete UML Diagrams

a use case diagram shows who uses the system and how, a class diagram shows the main objects in the backend and how they connect, and a sequence diagram traces one full package check from start to finish.

Use Case Diagram

Blueberry has two human actors. The **Developer** is the primary user and works directly inside the editor: detecting suggestions, viewing warnings, and configuring settings. **The Team Lead or Instructor** is a secondary user who mainly reviews the dashboard. PyPI Registry and GitHub are external systems the backend calls out to during verification. The 'Detect AI-suggested package name' use case includes the three verification use cases, which all feed into 'Compute explainable risk score' — every full check always runs all three verifications before a score is produced.

Figure 4 — Use Case Diagram

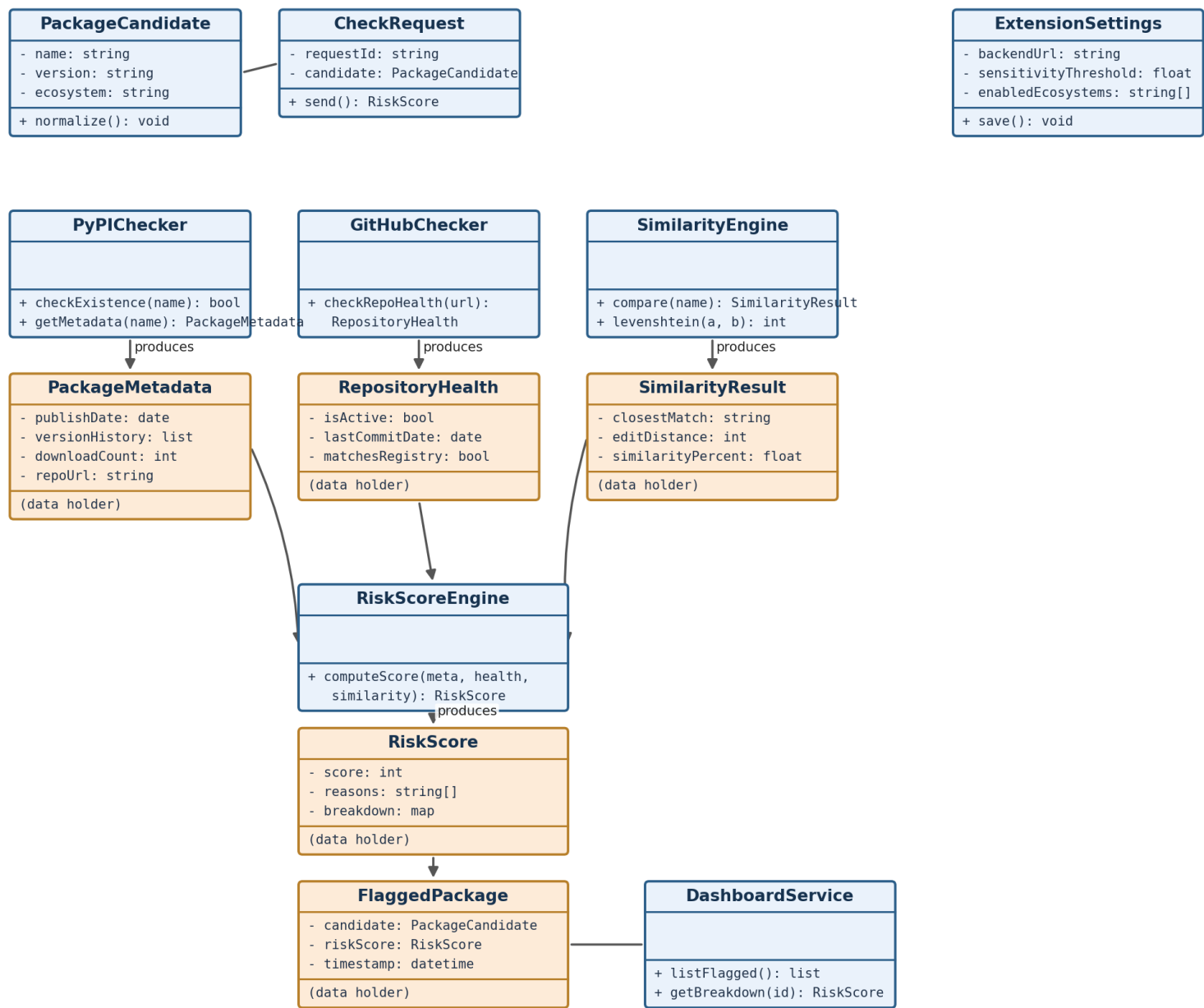


Class Diagram

The class diagram shows the main data and logic classes inside the backend. PackageCandidate and CheckRequest represent the package a developer's editor detected. PyPIChecker, GitHubChecker, and SimilarityEngine are three independent checker classes, each producing its own result object (PackageMetadata, RepositoryHealth, SimilarityResult). RiskScoreEngine combines all three results into one RiskScore, which is paired with the original candidate and a timestamp inside FlaggedPackage for

storage and later review. DashboardService reads these stored records for the dashboard, and ExtensionSettings holds the developer's configurable preferences.

Figure 5 — Class Diagram

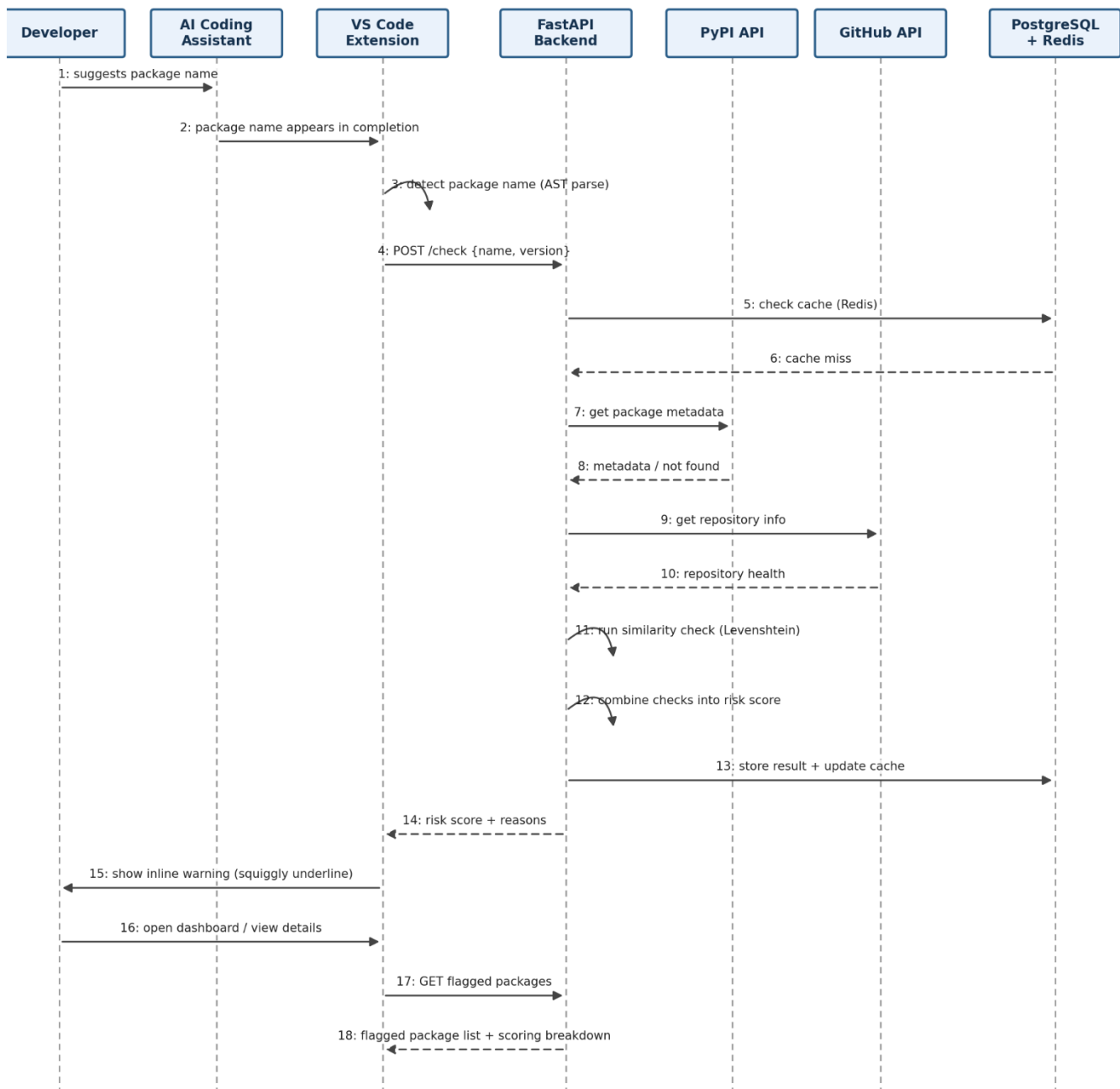


Sequence Diagram

The sequence diagram traces one full package check end to end: from an AI assistant suggesting a package, through detection, the four backend checks, storage and caching, and finally the developer seeing an inline warning and optionally opening the dashboard for more detail.

Figure 6 — Sequence Diagram: Package Suggestion to Risk Score Delivery

Figure 6 — Sequence Diagram: Package Suggestion to Risk Score Delivery

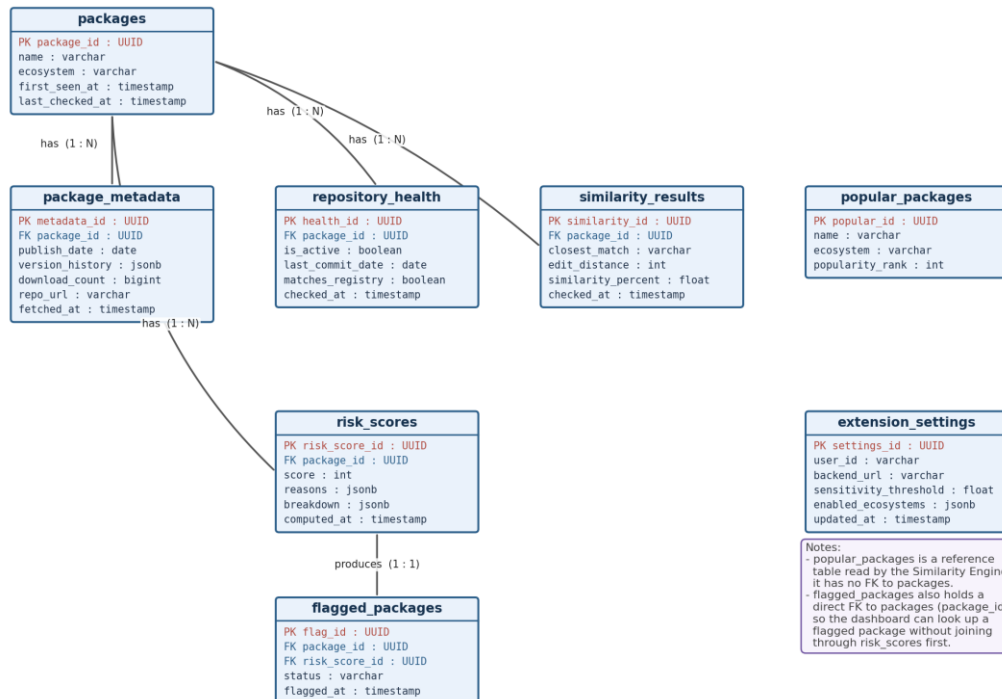


5. Database Design

Blueberry uses PostgreSQL as its persistent data store, exactly as fixed in the tech stack. The schema below turns the class diagram's data-holder classes (PackageMetadata, RepositoryHealth, SimilarityResult, RiskScore, FlaggedPackage) into real tables. Every check result is linked back to the package it belongs to through a shared package_id, so the full history of a package's checks can always be traced.

Entity-Relationship Diagram

Figure 7 — Entity-Relationship Diagram



Database Schema

The schema has eight tables. A central packages table holds one row per unique package name and ecosystem. Three tables hold the results of the three independent checks (package_metadata, repository_health, similarity_results), each linked to packages by a foreign key. risk_scores holds the combined, explainable score, and flagged_packages is the record shown on the dashboard, linking a package to its risk score with a timestamp and status. popular_packages is a standalone reference list used only for similarity comparisons, and extension_settings stores each developer's configuration.

1. packages — one row per package name + ecosystem ever seen.
2. package_metadata — PyPI check results for a package.
3. repository_health — GitHub check results for a package.
4. similarity_results — typosquat similarity check results for a package.
5. risk_scores — the combined, explainable score for a package.
6. flagged_packages — the record a developer or team lead reviews, linking a package to its risk score.
7. popular_packages — reference list of trusted, high-traffic package names.
8. extension_settings — each developer's saved extension preferences.

Table Descriptions

packages

Stores one row for every distinct package name and ecosystem that Blueberry has ever checked.

Column	Type	Description
package_id (PK)	UUID	Unique identifier for the package.
name	varchar	The package name as suggested by the AI assistant.
ecosystem	varchar	The package ecosystem — 'pypi' in this phase.
first_seen_at	timestamp	When Blueberry first encountered this package name.
last_checked_at	timestamp	When this package was last verified.

package_metadata

Stores the result of the PyPI Checker for a package.

Column	Type	Description
metadata_id (PK)	UUID	Unique identifier for this metadata record.
package_id (FK)	UUID	Links to the package this metadata belongs to.
publish_date	date	The date the package was first published on PyPI.
version_history	jsonb	List of published versions and their dates.
download_count	bigint	Reported download count, used as a trust signal.
repo_url	varchar	Source repository link listed on PyPI, if any.
fetched_at	timestamp	When this metadata was retrieved from PyPI.

repository_health

Stores the result of the GitHub Checker for a package.

Column	Type	Description
health_id (PK)	UUID	Unique identifier for this health record.
package_id (FK)	UUID	Links to the package this record belongs to.
is_active	boolean	Whether the repository shows recent activity.
last_commit_date	date	Date of the most recent commit found.
matches_registry	boolean	Whether the repository matches what PyPI lists.
checked_at	timestamp	When this repository check was performed.

similarity_results

Stores the result of the Similarity Engine for a package.

Column	Type	Description
similarity_id (PK)	UUID	Unique identifier for this similarity record.
package_id (FK)	UUID	Links to the package this record belongs to.
closest_match	varchar	The most similar popular package name found.
edit_distance	int	Levenshtein distance to the closest match.
similarity_percent	float	Similarity to the closest match, as a percentage.
checked_at	timestamp	When this similarity check was performed.

risk_scores

Stores the combined, explainable score produced by the Risk Score Engine.

Column	Type	Description
risk_score_id (PK)	UUID	Unique identifier for this score.
package_id (FK)	UUID	Links to the package this score belongs to.
score	int	The overall risk score.
reasons	jsonb	Plain-English reasons behind the score, e.g. 'no matching GitHub repository'.
breakdown	jsonb	How much each check (PyPI, GitHub, similarity) contributed.
computed_at	timestamp	When this score was computed.

flagged_packages

The record reviewed by a developer or team lead — links a package to its most recent risk score.

Column	Type	Description
flag_id (PK)	UUID	Unique identifier for this flagged entry.
package_id (FK)	UUID	The package that was flagged.
risk_score_id (FK)	UUID	The risk score that caused this package to be flagged.
status	varchar	'open', 'ignored', or 'resolved'.
flagged_at	timestamp	When this package was flagged.

popular_packages

A standalone reference table of trusted, high-traffic package names, used only by the Similarity Engine. It is not linked to the packages table by a foreign key.

Column	Type	Description
popular_id (PK)	UUID	Unique identifier for this reference entry.
name	varchar	A well-known, trusted package name.
ecosystem	varchar	The ecosystem this name belongs to — 'pypi' in this phase.
popularity_rank	int	Rough popularity ranking, used to prioritise comparisons.

extension_settings

Stores each developer's configuration for the VS Code extension.

Column	Type	Description
settings_id (PK)	UUID	Unique identifier for this settings record.
user_id	varchar	Identifies the developer these settings belong to.
backend_url	varchar	The FastAPI backend URL this extension talks to.
sensitivity_threshold	float	The risk score above which a warning is shown.
enabled_ecosystems	jsonb	Which package ecosystems are checked — PyPI only, in this phase.
updated_at	timestamp	When these settings were last changed.